Tablice wielowymiarowe

Inicjacja tablicy dwuwymiarowej

Jako tablicę dwuwymiarową możemy sobie wyobrazić planszę prostokątną składającą się z pewnej liczby wierszy i kolumn (indeksowanie rozpoczynamy od zera). Aby przypisać lub pobrać wartość do danej komórki, należy podać jej obie współrzędne.

Inicjacja tablicy polega na podaniu ilości wierszy i kolumn:

```
typ_elementów_tablicy nazwa_tablicy [ ilość wierszy ][ ilość kolumn ];
```

Przykład:

```
//stworzenie dwuwymiarowej tablicy liczb całkowitych, w sumie 100 komórek: 10x10.
int tab [ 10 ][ 10 ];
```

Nadanie wartości początkowych tablicy

Wartości początkowe tablicy możemy nadać przy jej deklaracji. Popatrzmy na przykład oparty na tablicy dwuwymiarowej liczb całkowitych:

```
//zauważmy, że mamy trzy wiersze, i dwie kolumny
int tab[3][2] = {{1,3},{4,5},{0,-1}};
```

Przy nadawaniu wartości tablicy można pominąć wartość w pierwszym nawiasie kwadratowym:

```
//wartość pierwszego nawiasu została pominięta
int tab[][2] = {{1,3},{4,5},{0,-1}};
```

Odwoływanie się do komórek w tablicy dwuwymiarowej

Aby odwołać się do każdej z komórek należy w nawiasach kwadratowych podać numer wiersza i kolumny komórki, do której się odwołujemy, pamiętając o tym, że numerujemy je od zera:

```
int tab[][2] = {{1,3},{4,5},{0,-1}}; //deklaracja i inicjacja tablicy dwuwymiarowej
cout<<tab[2][1]; //wyświetlenie wartości komórki znajdującej się w trzecim wierszu i w drugiej kolumnie (-1)
tab[0][0] = -100; //przypisanie do pierwszej komórki wartości -100
```

Przydzielanie pamięci na tablicę dwuwymiarową

Jeśli wielkość tablicy zależy od pewnych czynników, na przykład podajemy z klawiatury jej wielkość, lub ograniczenia stosu nie pozwalają nam na stworzenie odpowiednio dużej tablicy, to możemy przydzielić pamięć na nią dynamicznie (pobrać pamięć ze sterty):

```
int **tab, k, w;
        cout<<"Podaj liczbę wierszy i kolumn w tablicy: ";

        cin>>w>>k;

        tab = new int *[w]; //przydzielenie pamięci na w wierszy

        for(int i=0;i<w;i++)
                tab[i] = new int[k]; //przydzielenie dla każdego wiersza po k komórek

        //instrukcje własciwe programu

        //zwolnienie pamięci
        for(int i=0;i<w;i++)
                delete [] tab[i];

        delete [] *tab;
```

Warto przy okazji zauważyć, że liczba komórek w każdym wierszu może mieć różną wartość, czyli tablica dwuwymiarowa nie musi być prostokątna:

```
int **tab, k, w;
        cout<<"Podaj liczbę wierszy: ";

        cin>>w;

        tab = new int *[w]; //przydzielenie pamięci na w wierszy

        for(int i=0;i<w;i++)
        {
                cout<<"Podaj liczbę komórek w wierszu o numerze "<<i<<": ";
                cin>>k;
                tab[i] = new int[k]; //przydzielenie dla każdego wiersza po k komórek
        }

        //instrukcje własciwe programu

        //zwolnienie pamięci
        for(int i=0;i<w;i++)
                delete [] tab[i];

        delete [] *tab;
```

Przykład 1

Napisz program, który wykona transpozycję macierzy 4x5. Liczby generujemy losowo z przedziału [-9; 9]. Elementami macierzy są liczby całkowite.

Rozwiązanie

Macierz to obiekt, który doskonale nadaje się do przechowywania w tablicach dwuwymiarowych. Każda macierz składa się z pewnej ilości wierszy i kolumn. Przykładowa macierz spełniająca warunki zadania:

```
Przed transpozycja:
1 7 4 0 9
4 8 8 2 4
5 5 1 7 1
1 5 2 7 6
```

Transpozycja macierzy polega na zamianie wierszy z kolumnami. Powyższa macierz powinna wyglądać następująco:

```
Po transpozycji:
1 4 5 1
7 8 5 5
4 8 1 2
0 2 7 7
9 4 1 6
```

```
1   #include<iostream>
2   using namespace std;
3   int main()
4   {
5
6       int tab[4][5];
7
8       cout<<"Przed transpozycja:"<<endl;
9
10      for(int i=0;i<4;i++)
11      {
12          for(int j=0;j<5;j++)
13          {
14              //wygenerowanie liczb z zakresu [0-9]
15              // i wpisanie jej do tablicy
16              tab[i][j]=rand()%10;
17              //wyświetlenie elementu tablicy
18              cout<<tab[i][j]<<" ";
19          }
20          cout<<endl; // nowy wiersz
21      }
22
23      //transpozycja macierzy
24      cout<<"Po transpozycji: "<<endl;
25      for(int i=0;i<5;i++)
26      {
27          for(int j=0;j<4;j++)
28              cout<<tab[j][i]<<" ";
29
30      cout<<endl;
31      }
32
33      return 0;
34  }
```

Zerowanie tablicy wielowymiarowej

Jeśli chcemy wyzerować tablicę (ustawić wszystkie jej komórki na wartość 0, możemy to zrobić przy jej definiowaniu:

```
int tab[5][5] = {};
```

Taka konstrukcja zadziała tylko przy tworzeniu tablicy. Jeśli chcemy ponownie ją wyzerować musimy posłużyć się pętlami (w przypadku dwuwymiarowej tablicy będą dwie zagnieżdżone pętle):

```
1   #include<iostream>
2   using namespace std;
3
4
5   int main()
6   {   int tab[10][20];
7       for(int i=0;i<10;i++)
8           for(int j=0;j<20;j++)
9               tab[i][j]=0;
10
11      return 0;
12  }
```

Jeśli tablicę zadeklarujemy jako globalną, także zostanie wyzerowana:

```
1   #include<iostream>
2   using namespace std;
3   int tab[10][20]; //wyzerowana tablica
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7
8   return 0;}
```

Tablice wielowymiarowe

Tablice o większej liczbie wymiarów rzadko się stosuje. Sposób inicjacji, oraz operowania na tablicach tego typu jest analogiczny jak w przypadku tablic dwuwymiarowych.

Prześledźmy przykład tworzenia tablicy trójwymiarowej:

```
1   #include<iostream>
2   using namespace std;
3
4   int main(){
5   setlocale(LC_CTYPE, "Polish");
6
7       //deklaracja tablicy trójwymiarowej
8       //taka tablica posiada 3*4*5 = 60 komórek
9       int tab[3][4][5];
10      //przypisanie wartości 23 do pierwszej komórki
11      tab[0][0][0] = 23;
12
13      cout<<tab[0][0][0]<<endl;
14
15  return 0;}
```